

DAY 25

Database Fundamentals & CRUD

PostgreSQL (Supabase) + n8n Workflow Automation

A hands-on walkthrough of the four core database operations — Create, Read, Update, and Delete — implemented on a Supabase PostgreSQL **tickets** table and verified end-to-end through n8n automation workflows.

Project	petalnex-rag — Supabase (Production)
Table	public.tickets
Tools Used	PostgreSQL, Supabase Table Editor, n8n
Operations Covered	CREATE TABLE, INSERT, SELECT, UPDATE, DELETE

1 | CREATE TABLE

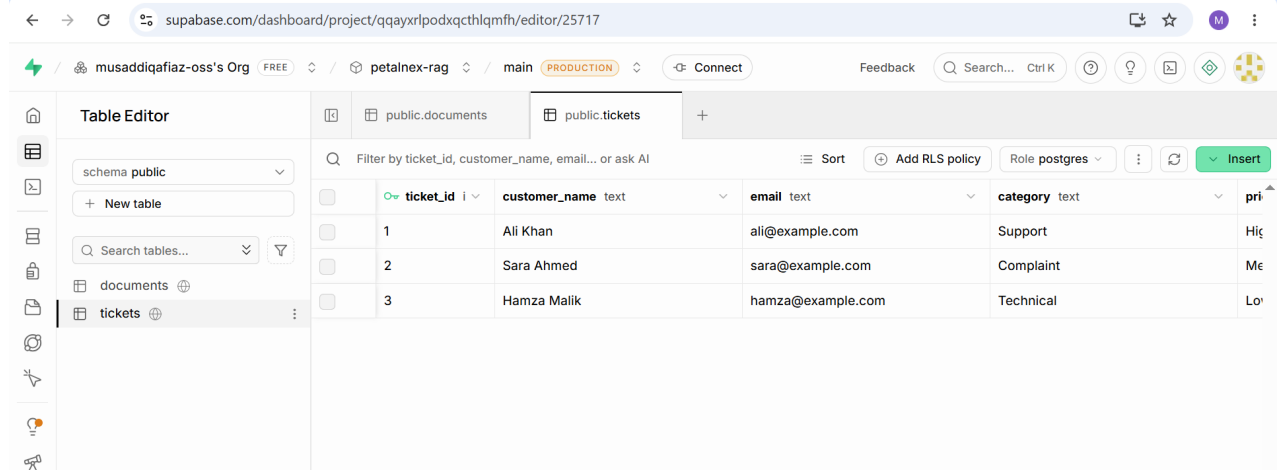
The **tickets** table is the foundation of the schema. It stores customer support tickets with an auto-incrementing primary key, contact details, classification fields, and an AI-generated summary.

```
CREATE TABLE tickets (  
  ticket_id BIGSERIAL PRIMARY KEY,  
  customer_name TEXT NOT NULL,  
  email TEXT NOT NULL,  
  category TEXT,  
  priority TEXT,  
  status TEXT DEFAULT 'Open',  
  ai_summary TEXT,  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

2 | CREATE — Insert Initial Records

Three seed tickets are inserted directly via SQL to populate the table with sample customer support cases.

```
INSERT INTO tickets  
  (customer_name, email, category, priority, status, ai_summary)  
VALUES  
  ('Ali Khan', 'ali@example.com', 'Support', 'High', 'Open',  
   'Customer cannot access the company portal'),  
  ('Sara Ahmed', 'sara@example.com', 'Complaint', 'Medium', 'Open',  
   'Customer reported a billing issue'),  
  ('Hamza Malik', 'hamza@example.com', 'Technical', 'Low', 'Open',  
   'Customer needs help configuring the application');
```



The screenshot shows the Supabase Table Editor interface. The left sidebar displays the project structure with 'public.tickets' selected. The main area shows a table with 3 rows of data. The table has columns: ticket_id, customer_name, email, category, priority, and status. The data is as follows:

ticket_id	customer_name	email	category	priority	status
1	Ali Khan	ali@example.com	Support	High	Open
2	Sara Ahmed	sara@example.com	Complaint	Medium	Open
3	Hamza Malik	hamza@example.com	Technical	Low	Open

Supabase Table Editor — tickets table after the initial 3-row INSERT

3 | READ — Select Records

The READ operation retrieves rows from the table. Below, the same operation is executed through an n8n **Postgres – Select rows from a table** node, returning the Hamza Malik ticket.

```
SELECT *  
FROM tickets;
```

The screenshot shows the n8n workflow editor interface. The workflow consists of two nodes: 'When clicking "Execute workflow"' and 'Select rows from a table'. The 'Select rows from a table' node is highlighted, and its execution logs are visible at the bottom. The logs show a successful execution in 1.279s. The output table displays the following data:

ticket_id	customer_name	email	category	priority
3	Hamza Malik	hamza@example.com	Technical	Low

A notification at the top right states: 'Node executed successfully'.

n8n workflow — "Select rows from a table" node executed successfully

4 | CREATE — Insert via n8n Automation

A new ticket for customer **Hina** is added using an n8n **Postgres – Insert rows in a table** node, demonstrating automated record creation outside the SQL editor.

```
INSERT INTO tickets
(customer_name, email, category, priority, status, ai_summary)
VALUES
('Hina', 'hina@gmail.com', 'Support', 'High', 'Open',
'Customer needs help with account access');
```

The screenshot shows the n8n workflow editor interface. The workflow consists of two nodes: 'When clicking "Execute workflow"' and 'Insert rows in a table'. The 'Insert rows in a table' node is highlighted, and its execution logs are visible at the bottom. The logs show a successful execution in 1.77s. The output table displays the following data:

ticket_id	customer_name	email	category	priority
4	hina	hina@gmail.com	Support	High

n8n workflow — "Insert rows in a table" node adding the Hina ticket (ticket_id 4)

The screenshot shows the Supabase Table Editor interface. On the left, a sidebar contains navigation icons and a list of tables: 'documents' and 'tickets'. The 'tickets' table is selected. The main area displays a table with the following data:

ticket_id	customer_name	email	category	priority
1	Ali Khan	ali@example.com	Support	High
2	Sara Ahmed	sara@example.com	Complaint	Medium
3	Hamza Malik	hamza@example.com	Technical	Low
4	Hina	hina@gmail.com	Support	High

At the bottom of the table, it indicates 'Page 1 of 1', '100 rows', and '4 records'. A green 'Insert' button is visible in the top right corner.

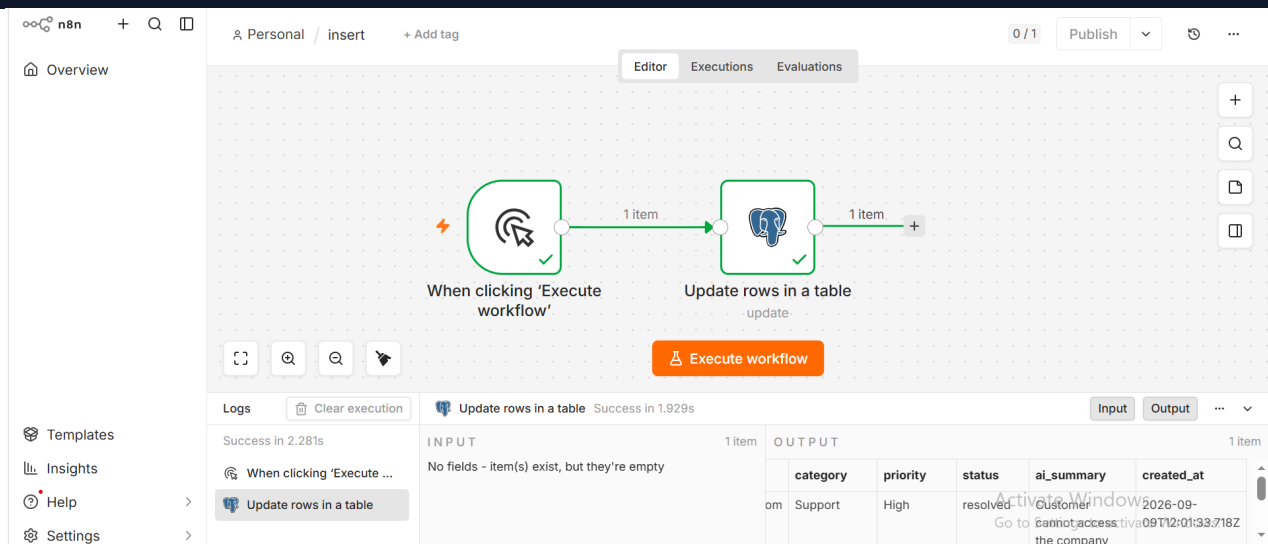
Supabase Table Editor — tickets table now shows 4 records after the automated insert

5 | UPDATE — Modify Ticket #1

Ticket ID 1 (Ali Khan) is marked as resolved. The n8n **Postgres – Update rows in a table** node performs the same change and returns the updated row with **status = resolved**.

```
UPDATE tickets
SET status = 'Resolved'
WHERE ticket_id = 1;

-- Verify the updated ticket
SELECT *
FROM tickets
WHERE ticket_id = 1;
```



n8n workflow — "Update rows in a table" node: ticket #1 status changed to "resolved"

6 | DELETE — Remove Test Ticket #2

Ticket ID 2 (Sara Ahmed) is removed from the table. The n8n **Postgres – Delete table or rows** node confirms the deletion with **success: true**, and the Supabase table now reflects the final, cleaned-up dataset.

```
DELETE FROM tickets
WHERE ticket_id = 2;

-- Verify remaining tickets
SELECT *
FROM tickets;
```

The screenshot shows the n8n workflow editor interface. The workflow consists of two nodes: 'When clicking "Execute workflow"' and 'Delete table or rows'. The 'Delete table or rows' node is highlighted, and its execution logs are visible. The logs show a successful execution with the message 'Delete table or rows Success in 1.35s'. The output of the node is 'success: true'.

Logs: Success in 1.549s

When clicking "Execute ..."

Delete table or rows

INPUT: No fields - item(s) exist, but they're empty

OUTPUT: success: true

n8n workflow — "Delete table or rows" node executed successfully

The screenshot shows the Supabase Table Editor interface. The table 'tickets' is selected, and its data is displayed. The table has 3 records remaining after the deletion of ticket ID 2.

ticket_id	customer_name	email	category	priority
1	Ali Khan	ali@example.com	Support	High
3	Hamza Malik	hamza@example.com	Technical	Low
4	hina	hina@gmail.com	Support	High

Page 1 of 1 100 rows 3 records

Supabase Table Editor — final state: 3 records remain (tickets 1, 3, 4) after DELETE

CRUD Operation Summary

Operation	SQL Command	n8n Node Used	Result
CREATE	INSERT	Insert rows in a table	Ticket #4 (Hina) added
READ	SELECT	Select rows from a table	Ticket #3 (Hamza) retrieved
UPDATE	UPDATE	Update rows in a table	Ticket #1 status → Resolved
DELETE	DELETE	Delete table or rows	Ticket #2 (Sara) removed

Key takeaway: Every core CRUD operation was validated twice — once directly via raw SQL against the PostgreSQL/Supabase database, and once through an equivalent n8n Postgres node — confirming that the automation layer behaves identically to manual SQL execution.